What Survives When You Compress a Recursive Reasoner for the Edge?

Pearse Jim[1]*, **Steven Kolawole**[2,1]*, **Opegbemi M. Busoye**[1], **Glory Bagai**[1], **Virginia Smith**[2]

[1]ML Collective, [2]Carnegie Mellon University

arXiv:2606.26488v2 [cs.LG] 30 Sep 2026

Abstract

Recursive reasoning models solve hard structured tasks with a few million parameters by iterating a latent state, but deploying them on edge hardware means compressing them – and quantization noise compounds across recursive cycles rather than accumulating over output tokens, so single-pass intuitions fail. Here, we ask what survives. Across a full precision sweep and three tasks, aggressive compression preserves *local* prediction but destroys *global* reasoning: pruning, distillation, and linear attention drive puzzle-exact accuracy to zero while cell accuracy holds. Naïve INT4 is different: its damage is architectural, striking MLP-mixing recursion but not attention on the same task, and per-channel calibration reverses it without retraining, where quantization-aware training does not. We also introduce carry-trajectory fidelity, the cosine similarity to the full-precision reasoning path, as a label-free signal that predicts both the damage and the recovery before any task evaluation, with a threshold that transfers across tasks. The result is a deployment recipe: flash-streamed embeddings remove a 99.4 MB bottleneck, one recursive cycle matches full-depth accuracy at 6× fewer FLOPs, and calibrated INT4 brings the backbone to 3.3 MB, inside a 4 MB budget.

1 Introduction

The Tiny Recursive Model (TRM) solves abstract reasoning puzzles with only 6.83M parameters (Jolicoeur-Martineau, 2025)—44.6% on ARC-AGI-1 with 1000-augmentation test-time voting (36.00% single-pass in our harness)—competitive with larger autoregressive systems on a benchmark designed to resist pattern matching (Chollet, 2019). It does so by iterating a latent carry state across recursive cycles, a mechanism shared by Universal Transformers (Dehghani et al., 2019) and depth-recurrent language models (Geiping et al., 2025; Wang et al., 2025; Hao et al., 2025; Kohli et al., 2026). Yet TRM weighs ∼126 MB, more than an order of magnitude beyond the kB-to-MB SRAM of the microcontroller tier we target. Deployment there requires compression. When such a model is squeezed, what survives and what breaks?

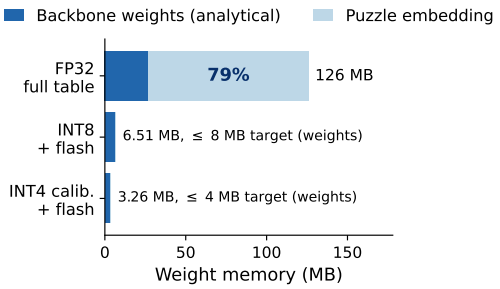


Figure 1: Backbone-weight memory by configuration (analytical, weights-only). The FP32 embedding table (99.4 MB) accounts for 79% of the ∼126 MB baseline. Flash loading reduces the active embedding to 2 KB, isolating backbone compression. INT8 + flash reaches 6.51 MB and calibrated INT4 + flash 3.26 MB, the latter within the 4 MB microcontroller budget. Both are weights-only; with activations and carry state the ARC/Maze streaming peak is 9.51 MB (Table 4).

While single-pass transformer quantization is well-studied (noise accumulates over output tokens, and INT8 deployment is standard) (Nagel et al., 2021; Dettmers et al., 2022; Han et al., 2016), recursive models introduce a different failure mode: quantization noise compounds across H recursive cycles at each carry-state update. Whether this compounding degrades reasoning has not been studied. While quantization harms single-pass reasoning chains (Li et al., 2025) and emergent abilities collapse below 4-bit (Liu et al., 2024), depth and precision are independent in those systems. In recursive models, they are not: the optimal operating point depends jointly on bit-width *and* depth.

We present the first systematic compression study of a recursive reasoning model under hard

* Equal Contribution.

SRAM constraints (Lin et al., 2020; Abushahla et al., 2025). We sweep precision (FP32, FP16, INT8, INT4) and depth across three tasks (ARC-AGI-1, Maze-Hard, Sudoku-Extreme) and two architectures (TRM and HRM). As shown in Figure 1, the dominant memory bottleneck is the 99.4 MB puzzle embedding table, which we resolve via 2 KB flash loading, isolating backbone compression. Our primary contributions are:

1. We establish a **depth–precision crossover** with two separable effects: a single recursive cycle matches full-depth accuracy at $6\times$ fewer FLOPs—a depth effect (FP32 at $H{=}1$ scores 35.75% vs. INT8's 35.25%)—while INT8 adds a $4\times$ weight-memory reduction at no accuracy cost, an interaction absent from single-pass quantization (§4.2).
2. We identify a **compositional collapse**: under pruning, distillation, and linear-attention substitution, local token accuracy survives while global puzzle accuracy falls to zero; naïve INT4's harm is architecture-dependent, and an attention-vs-MLP-mixing ablation shows the fragility is architectural, not task-intrinsic (§4.3).
3. We introduce **carry-trajectory fidelity**, a label-free detector that grades this damage and predicts its recovery from the model's own hidden states, with no task labels (§4.5).
4. We derive and benchmark a **deployment recipe**: per-channel calibrated INT4 weights reach 3.26 MB, within a 4 MB microcontroller budget, with latency, memory, and file size measured directly on three Snapdragon mobile platforms (§4.4, 4.7).

2 Related Work

Overthinking in recursive models is named but not detectable. Recurrent and weight-tied architectures are a parameter-efficient path to algorithmic reasoning (Dehghani et al., 2019), and recent depth-recurrent language models show iterative latent refinement scales favorably (Geiping et al., 2025; Wang et al., 2025; Hao et al., 2025; Kohli et al., 2026). Bansal et al. (2022) name their central failure—*overthinking*, where too many iterations corrupt the instance—but detect it only by full task evaluation; Du et al. (2026) and Lu et al. (2025) find correctness signals in latent trajectories, yet none give a signal usable *during inference* without labels. Our carry-trajectory fidelity, grounded in Jiang et al. (2025)'s link between hidden-state cosine similarity and representational saturation, predicts collapse under quantization with no task evaluation.

Quantization-as-regularization has an uncharacterized depth dimension. Quantization noise can act as a regulariser – pushing toward flatter minima with a model-specific optimal level (Askari-Hemmat et al., 2024, 2022), robust under distribution shift and limited data (Javed et al., 2025; Tallec et al., 2023), sometimes letting INT8 beat FP32 (Zhang et al., 2025) – which explains INT8 parity in single-pass models. None predicts recursive behavior, where the same noise is applied across H carry-state updates and beneficial regularization at shallow depth becomes destructive compounding at depth. We characterize this depth–precision interaction directly.

Compressed reasoning fails early; low-bit recovery needs the right recipe. On reasoning chains, PTQ induces early failures that cascade (Li et al., 2025), with a precision floor near 4-bit (Liu et al., 2024) and W8A8 as the lossless threshold for small models (Liu et al., 2025); consistent with our INT8 finding, but for single-pass decoding. Standard QAT recipes for reasoning models favor distillation/KD objectives and PTQ initialization (Lv et al., 2026); our QAT comparison follows that recipe (calibrated initialization, CE and KD objectives, three learning rates) and finds that further token-level training degrades an already-calibrated model (§4.6), so the quantizer is the reliable recovery path in our setting.

Edge NLP deployment motivates but does not resolve these questions. Foundational compression (Han et al., 2016), the finding that pruning degrades small LMs more than quantization (Zhou et al., 2025) (consistent with our pruning results), INT8 as the deployment-native format (Dettmers et al., 2022; Kim et al., 2021), sub-megabyte MCU inference via hardware co-design (Lin et al., 2020), MCU-quantization surveys at our 4–8 MB constraints (Abushahla et al., 2025), and on-device NLP pipelines (Bohdal et al., 2025): none studies recursive reasoning models or the depth–precision interplay. No prior work compresses a recursive reasoning model; the closest (Li et al., 2025) addresses single-pass decoding and lacks the recur-

sive carry-state structure we study.

3 Background and Experimental Setup

3.1 Tiny Recursive Model (TRM)

TRM (Jolicoeur-Martineau, 2025) is a weight-tied recursive transformer that solves structured reasoning tasks through iterative latent refinement rather than autoregressive generation. Let $\mathcal{V}$ be a finite vocabulary and $X \in \mathcal{V}^{T_x}$ be an input sequence. TRM maintains a latent carry state updated across two nested loops.

Given an initial answer embedding $Y_0 \in \mathbb{R}^{T_y \times d}$, the model maintains a carry state $Z_h \in \mathbb{R}^{T_z \times d}$ across outer cycles $h \in \{1, \ldots, H\}$. At each outer cycle, an inner loop executes L refinement steps. A shared two-layer transformer block f_θ updates the state:

$$Z_h^{(l)} = f_\theta\Big([X\,;\, Y_{h-1}\,;\, Z_h^{(l-1)}]\Big) \qquad (1)$$

where $[\cdot;\cdot]$ denotes sequence concatenation. After L inner steps, the outer loop advances via:

$$Y_h,\; Z_{h+1}^{(0)} = g_\phi\Big(Z_h^{(L)}\Big) \qquad (2)$$

A one-step gradient approximation during training keeps memory $\mathcal{O}(1)$ with respect to recursive depth. The total number of refinement steps is $H \times n_{\text{sup}}$, where n_{sup} is the number of outer-loop supervision steps. At the default ARC-AGI-1 configuration ($H = 3$, $n_{\text{sup}} = 16$), TRM executes 48 refinement steps, incurring ∼3,000 GFLOPs per puzzle over a 900-token sequence ($\mathcal{O}(n^2)$ attention).

We study two architectural variants: **TRM-Attention**, which uses multi-head self-attention over the full token sequence, and **TRM-MLP-Mixing**, which replaces attention with token-mixing MLP layers. These variants differ in how relational structure is encoded across refinement steps, which has consequences for compression sensitivity (§4.3).

3.2 Tasks and Datasets

ARC-AGI-1. Abstract visual reasoning requiring detection of transformation rules across grid inputs (Chollet, 2019). We evaluate on the ARC-AGI-1 public evaluation set (400 tasks). TRM uses TRM-Attention with a 50,911-entry puzzle embedding table (99.4 MB in FP32) that provides puzzle-specific context and, in its published protocol, test-time augmentation (1,000 augmentations per puzzle) with multi-pass majority voting; our evaluations run single-pass (no TTA) unless noted. Parameters: 6.83M.

Task	Puzzle-exact % ↑			
	FP32	INT8	INT4	INT4 cal.
Sudoku, TRM-MLP	74.6	74.7	6.4	74.5
Sudoku, TRM-Attn	73.1	72.8	68.3	72.3
Maze, TRM-Attn	87.0	87.1	86.2	86.7

Table 1: Puzzle-exact accuracy (%) at the primary protocol (n_{sup}=16, fp32 forward). Sudoku: random n=2,000 subset (both architectures on the same subset); Maze: full 1,000-maze test set. INT4 is naïve per-tensor; INT4 cal. is per-channel calibrated.

Maze-Hard. Path-finding requiring exact end-to-end traversal over topological maps. Uses TRM-Attention without an embedding table or TTA. Parameters: 6.82M.

Sudoku-Extreme. Constraint satisfaction over 9×9 grids with hard row, column, and block exclusions. Uses TRM-MLP-Mixing without an embedding table. Parameters: 5.03M.

We report two metrics throughout. **Cell accuracy** is the fraction of correct grid cells predictions. **Puzzle exact match** is the fraction of puzzles solved with zero errors. These metrics can diverge substantially: a model may predict nearly correct tokens (high cell accuracy) while satisfying zero global constraints (zero puzzle exact), as observed under several compression interventions (§4.6).

3.3 Evaluation Protocol

Headline evaluations use TRM's test-time default: n_{sup}=16, an fp32 forward pass, and random test subsets (Table 1). We call this the primary protocol. The wider sweeps and ablations, marked at point of use, run a lighter secondary protocol (n_{sup}=10, bf16 forward, first-N subsets) whose absolute accuracies sit a few points lower. The two protocols agree on every ordering this paper reports.

At this protocol naïve INT4 costs MLP-mixing 68.2 pp against 4.8 pp for attention on the same subset (Table 1).

3.4 Deployment Targets

The compression goal is on-device inference with no cloud dependency under hard SRAM constraints. Our target is the microcontroller tier: kB-to-MB SRAM, no DRAM, Cortex-M class, where the deployed state of the art is 1–2 MB CNNs (Lin et al., 2020) and nothing reasoning-capable is deployed at all. We use two budgets as reference

points on the way there, 8 MB (mobile SoC SRAM, e.g. Cortex-A55) and 4 MB (MCU, e.g. Cortex-M55); the FP32 baseline (∼126 MB total: 26.7 MB backbone + 99.4 MB embeddings) exceeds both by an order of magnitude.

Measurement cannot reach the target tier directly: no MCU toolchain runs these graphs today, so the on-device latency, memory, and model sizes of §4.7 come from three Snapdragon platforms via Qualcomm AI Hub (App. B), which are mobile-class. Everything we report at the 4 MB budget is therefore an analytical, weights-only projection (Table 4).

3.5 Compression Methods

Post-training quantization (PTQ). We apply PTQ directly to the pretrained FP32 checkpoint, without retraining. **FP16** halves memory via standard half-precision casting. **INT8** uses per-channel scale calibration via `bitsandbytes` (Dettmers et al., 2022) - the standard format for hardware with integer acceleration units. **INT4 naïve** applies per-tensor PTQ without calibration. **INT4 calibrated** applies per-channel asymmetric PTQ (min/max range per output channel), again with no fine-tuning; it recovers most of the accuracy that naïve INT4 destroys (§4.4).

Structured pruning and knowledge distillation were also evaluated. Both collapse puzzle-level accuracy across all tasks while partially preserving cell accuracy (§4.6). We read this as a consequence of the model's compactness: at 5–7M parameters there is too little redundancy to absorb pruning, and a single-pass student cannot capture the recursive computation.

Embedding compression. For ARC-AGI-1, we separately evaluate three strategies for the 99.4 MB puzzle embedding table: INT8 quantization of the full table, SVD rank-16 factorization, and single-puzzle flash loading (streaming only the 2 KB row for the current puzzle at inference time).

3.6 Cycle Reduction

H (outer cycles) and n_{sup} (the number of outer-loop supervision steps) together determine total recursion depth: $H = 3$, $n_{\text{sup}} = 16$ means 48 sequential refinement steps on ARC-AGI-1. We sweep $(H, n_{\text{sup}}) \in \{(1, 1\text{–}16),\ (3, 1\text{–}16)\}$ for ARC-AGI-1, and $(H, n_{\text{sup}}) \in \{(1\text{–}4, 1\text{–}10)\}$ for Maze-Hard and Sudoku-Extreme. FLOPs scale linearly with total steps: at $H = 1$, $n_{\text{sup}} = 8$, the per-puzzle cost falls from ∼3,000 GFLOPs to ∼500 GFLOPs on ARC-AGI-1.

3.7 Carry-State Diagnostics

We use the recursive carry state to build a label-free signal for reasoning collapse under compression. Let $Z_h \in \mathbb{R}^{T_z \times d}$ be the carry state after recursive call h, with $z_{h,i}$ its i-th token vector.

Consecutive similarity (baseline). The cosine similarity between successive carry states,

$$s_h = \frac{1}{T_z} \sum_{i=1}^{T_z} \frac{z_{h,i} \cdot z_{h-1,i}}{\|z_{h,i}\|_2 \, \|z_{h-1,i}\|_2}, \tag{3}$$

measures per-step refinement (low s_h) versus convergence to a near-fixed point (high s_h), following Jiang et al. (2025)'s finding that hidden-state cosine similarity tracks representational saturation. We find this signal is nearly invariant to precision (§4.5) and thus a poor failure detector.

Carry-trajectory fidelity (primary). The signal we recommend compares the compressed trajectory to the full-precision one. Let Z_H^q and Z_H^{fp32} be the final carry states of the quantized and FP32 models on the same input. Fidelity is

$$\phi = \frac{1}{T_z} \sum_{i=1}^{T_z} \frac{z_{H,i}^q \cdot z_{H,i}^{\text{fp32}}}{\|z_{H,i}^q\|_2 \, \|z_{H,i}^{\text{fp32}}\|_2}. \tag{4}$$

ϕ quantifies how far quantization pushes the reasoning trajectory off the full-precision path. It requires only the FP32 reference (available before deployment) and no task labels, and—unlike s_h—is graded and monotonic with the eventual accuracy loss (§4.5).

3.8 Quantization-Aware Training and Recovery

As a probe of whether a token-level objective can repair compression-induced reasoning loss, we run QAT from the calibrated-INT4 checkpoint: 1,000 steps at batch 32, under cross-entropy and token-distillation objectives at three learning rates each, with the deep-supervision ACT loop (§4.6). As the alternative recovery path we apply per-channel calibrated INT4 (a post-training quantizer with no fine-tuning) and report both its accuracy and its carry-trajectory fidelity (§4.4).

Strategy	SRAM ↓	Accuracy ↑
INT8 quant. (full table)	24.9 MB	no loss
SVD rank-16 factorization	3.2 MB	lossy ($\cos = 0.94$)
Single-puzzle flash loading	2 KB	no loss

Table 2: Embedding compression strategies on ARC-AGI-1. Only flash loading brings the embedding within the deployment budgets without loss; the backbone and the activation-inclusive streaming peak are accounted separately (§4.7, Table 5).

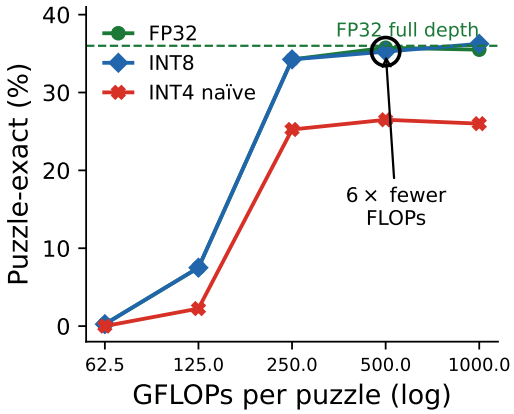


Figure 2: ARC-AGI-1 efficiency frontier (single-pass). One recursive cycle reaches full-depth accuracy at 500 GFLOPs (⋆)—a 6× FLOPs reduction attributable to depth: FP32 at the same depth scores 35.75%.

4 Results

4.1 The Embedding Table Dominates the Memory Footprint

The ARC-AGI-1 puzzle embedding table occupies 99.4 MB of the ∼126 MB FP32 footprint, against 26.7 MB for the transformer backbone (Table 2). Three strategies were evaluated for embedding compression.

Only flash loading fits the deployment targets while preserving accuracy (Table 2): the 2 KB row for the current puzzle is streamed from flash at inference time, decoupling the embedding from the backbone compression problem. Figure 1 visualizes the resulting footprints. With the embedding handled, the backbone remains, and with it two levers: precision and recursion depth.

4.2 INT8 with Reduced Recursion Matches Full-Depth FP32

INT8 is lossless across both architectural variants and all three tasks (Table 8, Appendix C): per-channel calibration holds quantization error within what the recursive refinement process tolerates. The key efficiency gain therefore comes from cycle reduction, not precision: FP32 at $H{=}1$, $n_{\text{sup}}{=}8$ already scores 35.75% (Table 9), and INT8 at the same depth achieves 35.25% puzzle exact on ARC-AGI-1, single-pass (no test-time augmentation),[1] within 0.75 pp of the 36.00% full-depth baseline at 6× fewer FLOPs (Figure 2). The two effects are separable: depth reduction supplies the 6× FLOPs saving, while INT8 supplies a 4× weight-memory reduction at no accuracy cost.

Across the full depth sweep (Appendices E and D), $H{=}1$ brackets the 36.00% baseline from 34.25% ($n_{\text{sup}}{=}4$) to 36.25% ($n_{\text{sup}}{=}16$). The reduction holds on Maze-Hard (INT8 at $H{=}1$, $n_{\text{sup}}{=}8$ stays within 0.10 pp at 3.75× fewer FLOPs). Sudoku-Extreme is the exception: its accuracy keeps climbing with depth within the swept range (best at $H{=}4$, $n_{\text{sup}}{\leq}10$) and INT8 stays lossless there, so its full recursion cannot be cheaply cut (Appendices F, G).

Dynamic quantization preserves full-precision accuracy on every task, but NPU accelerators require static, pre-allocated tensor boundaries; the static path's task-dependent penalties, and the operator exclusions Sudoku requires, are in Appendix B.2. INT8, then, is safe everywhere. INT4 is not, and where it breaks turns out to be the more informative finding.

4.3 Compression Fragility Is Architectural, Not Task-Intrinsic

The INT4 damage follows the architecture, not the task: the TRM-Attention tasks (ARC, Maze) tolerate naïve INT4 while TRM-MLP-Mixing (Sudoku) collapses (Table 8). This looks like a task-precision effect; a same-task ablation shows it is not (Figure 3; full grid in Appendix H).

Under naïve INT4, Maze-Hard loses 0.8 pp and ARC-AGI-1 loses 10.1 pp puzzle exact while keeping 89.2% cell accuracy, but Sudoku-Extreme collapses by 68.2 pp (primary protocol, Table 1; the official-checkpoint ARC figures are in §4.4). Since Sudoku was tested only with MLP-mixing, we rerun it with attention: TRM-Attention, at nearly the same full-precision accuracy on the same subset (73.1% vs. 74.6%), retains 68.3% under naïve

[1] All reported ARC accuracies are single-pass: the evaluation's fast mode discarded the augmentations, so no augment-and-vote protocol was applied. TRM's published 44.6% uses 1000-augmentation TTA and is not directly comparable. App. B compares the notebook harness with on-device execution.

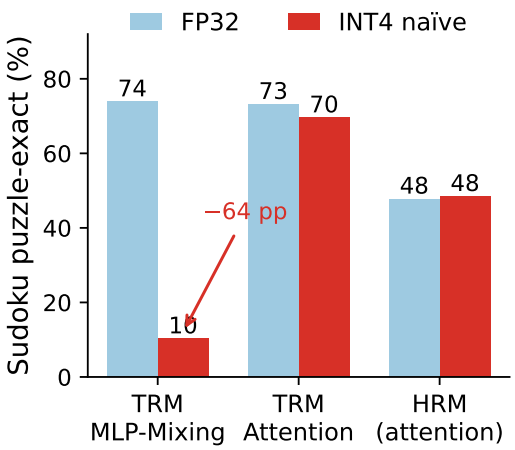


Figure 3: Architecture ablation on Sudoku-Extreme: full precision vs. naïve INT4 puzzle-exact for three recursive architectures on the *same* task (secondary protocol; the primary protocol widens the gap to −68 pp, Table 1). Only TRM-MLP-Mixing collapses; TRM-Attention (at the same FP32 accuracy) and HRM (a different model family) survive; the fragility follows the MLP-mixing token mixer.

INT4, and HRM (Wang et al., 2025) is also robust, while only MLP-mixing collapses (Table 15). The accuracy match rules out a headroom confound, locating the fragility in the token mixer. HRM, evaluated only on Sudoku and at a lower FP32 baseline, corroborates the pattern in a second model family rather than serving as a controlled comparison.

A weight-level mechanism. Per-output-channel dynamic range explains why the token mixer specifically. On the two Sudoku checkpoints the token-mixing projections reach a 20.7× inter-channel spread, against 11.1× for the worst attention projection. Figure 4 shows the underlying distributions and extends the comparison to the deployment models: within the same network the token mixer's spread is nearly three times the channel MLP's, and both attention models sit lower still. The token mixer is the outlier on every comparison available to us, within a network and across them. Under a single per-tensor INT4 scale, the widest-spread channels dominate the quantization grid and narrow channels collapse into one or two bins. Per-channel calibration repairs exactly this, and the recovery of §4.4 follows. A failure rooted in per-channel spread implies a per-channel fix, which the next section tests.

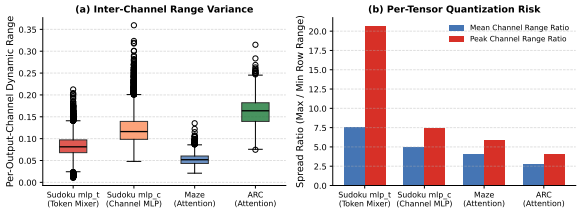


Figure 4: Per-output-channel weight range by projection type, across the deployment models. (a) The distribution for each projection. (b) The ratio of widest to narrowest channel range — what a single per-tensor scale must span. Sudoku's token mixer (`mlp_t`) is the outlier; its TRM-Attention counterpart, used as the same-task control in §4.3, is not plotted.

Variant	pass@1 ↑	Solved ↑	Cell ↑	Fidelity ↑
FP32	29.5%	115	91.5%	1.000
INT8 per-channel	30.0%	117	91.6%	1.000
INT4 naïve	19.4%	74	89.2%	0.765
INT4 calibrated	29.4%	114	91.6%	0.942

Table 3: Official TRM ARC-AGI-1 checkpoint, single-pass, all 400 evaluation tasks. pass@1 is the ARC per-task score, with partial credit over a task's multiple test inputs (419 test grids across the 400 tasks); Solved counts tasks with every test input correct. Calibration recovers ARC to within 0.1 pp of FP32; the fidelity column is a fully held-out test of the §4.5 threshold, which was never calibrated on ARC or on this checkpoint.

4.4 Calibrated INT4 Recovers Constraint Reasoning

The catastrophic Sudoku-INT4 result is specific to *naïve per-tensor* quantization. Replacing it with per-channel calibrated INT4 (still 4-bit weights, still post-training, no fine-tuning) recovers essentially all of the lost accuracy: at the primary protocol, naïve INT4 drops Sudoku to 6.4% puzzle exact while calibrated INT4 reaches 74.5%, within 0.1 pp of the 74.6% FP32 baseline on the same subset (Table 1; secondary-protocol figures in App. I). The carry-trajectory fidelity diagnostic tracks this recovery with no task evaluation: fidelity rises from 0.35 (naïve, broken) to 0.89 (calibrated, recovered).

Measured on the official ARC checkpoint. The same recovery holds on ARC-AGI-1, measured on the official TRM checkpoint (arcprize verification release; embedding table verified against the rebuilt official dataset, 876,406 identifiers), single-pass over all 400 evaluation tasks:

Calibration recovers ARC to within 0.1 pp of FP32 on a checkpoint anyone can download, and the fidelity column doubles as the held-out thresh-

old test cited in §4.5.

This reframes the deployment floor: for constraint-satisfaction reasoning it is not INT8 but *calibrated* INT4, provided the quantizer is per-channel. Naïve INT4 remains unusable, and the diagnostic distinguishes the two before deployment.

4.5 Carry-Trajectory Fidelity Predicts Collapse and Recovery

The damage in §4.3 and its repair in §4.4 were both established by running the task. We now ask whether either can be predicted without doing so: a label-free signal that, before any task evaluation, tells whether a compression preserves or destroys reasoning. The obvious candidate (cosine similarity between consecutive carry states) does not work: it is task-specific but nearly invariant to precision, changing by ≤ 0.064 from FP32 to INT4 on every task (App. I), so it cannot separate harmless from harmful compression.

The signal that does work is *carry-trajectory fidelity*: the cosine similarity between the quantized model's final carry state and the full-precision model's final carry state on the same input (§3.7). It measures how far quantization pushes the reasoning trajectory off the full-precision path, requires only the FP32 reference (available before deployment), and uses no task labels. Fidelity is graded and monotonic with damage (Figure 5; full data Appendix I): it stays ≥ 0.99 wherever INT4 is harmless (Maze, and INT8 on every task) and falls in proportion to the accuracy loss, moderately on ARC and catastrophically on the Sudoku collapse of §4.3 (Table 17). It tracks the calibrated-INT4 *recovery* of §4.4 in the same way, rising as accuracy returns.

The threshold transfers across tasks. We validate fidelity as a predictor on a 50-configuration grid spanning bit-widths, quantizer granularities and partial-layer sweeps across three model–task pairs (Appendix I); 8 configurations break, defined as a >10 pp puzzle-exact drop. Spearman ρ between fidelity and accuracy damage is -0.75 overall, fidelity separates broken from surviving configurations without error on this grid (AUC 1.00), and the dose–response is monotone (Figure 5). The threshold question now has a held-out answer: calibrated on Sudoku alone ($\tau = 0.83$), it transfers to Maze without error. The official ARC checkpoint of §4.4 is a third, fully held-out test, on which the threshold was never calibrated: it flags naïve

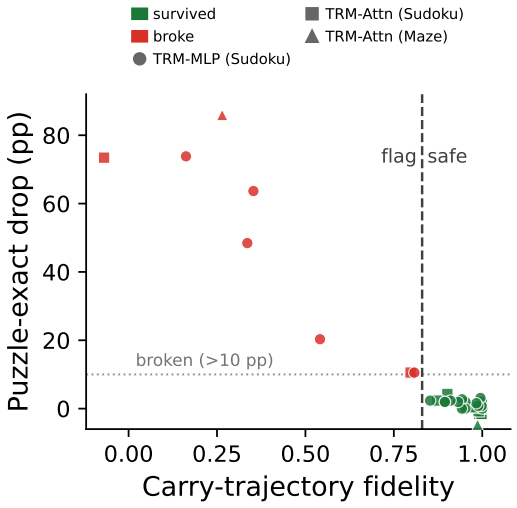


Figure 5: Carry-trajectory fidelity is a label-free pass/fail predictor. Each point is one of the 50 compression configurations in the validation grid (§4.5): bit-widths 3–8, three quantizer granularities and partial-layer sweeps, across three model–task pairs. Colour marks whether the configuration broke, defined as losing more than 10 pp puzzle-exact. The dashed line is the threshold τ=0.83 calibrated on Sudoku alone, which separates the two classes on held-out Maze without error.

INT4 and clears calibrated INT4 (fidelity column of Table 3). An L2 trajectory-divergence baseline computed from the same captures is equally predictive on this grid; fidelity's advantage is being bounded and scale-free across models, not higher discrimination. Quantization damage is therefore both repairable and predictable in advance. Neither holds for the remaining compression routes.

4.6 Cell and Puzzle Accuracy Decouple Under Compression

A pattern recurs across our compression interventions: cell accuracy and puzzle-exact accuracy decouple. A model can predict most individual cells correctly while satisfying zero global constraints—the signature of a *compositional-correctness* breakdown that leaves local token prediction intact. We observe it under three distinct interventions:

On **knowledge distillation**, a single-cycle student (13–15% of teacher parameters) reaches 87.3% (Maze) / 54.8% (Sudoku) cell accuracy but 0.0% puzzle exact (Appendix J, Tables 19 and 20). With **structured pruning** at 25% sparsity, cell accuracy partially survives (86.4% Maze, 50.0% Sudoku) while puzzle exact is 0.0% on every task. With **linear-attention approximation** on Maze, re-

placing softmax attention holds cell accuracy at 87.5% but collapses puzzle exact to 0.0% (Appendix K).

On **quantization-aware training**, we ran the credible version: training from the calibrated-INT4 Sudoku checkpoint under both cross-entropy and distillation objectives, at three learning rates, with converging training losses (Appendix L). Every configuration *degraded* held-out accuracy relative to its initialization (Table 21, Appendix L).

Within our training budget, then, a token-level objective does not repair the model it starts from, while per-channel calibration alone lands within 0.3 pp of FP32 on all three tasks (Tables 1 and 3). A shorter-budget run reproduces the same split: high cell accuracy, zero puzzle exact. We do not claim a token-level objective cannot recover reasoning at a larger budget; we claim it is not needed.

The common cause of the decoupling is the objective: cross-entropy and token-distillation losses reward each cell independently, so a model that emits plausible cells without satisfying the puzzle is scored identically to one that solves it. This is not specific to TRM—it applies to any structured-prediction setting where correctness is compositional. Our contribution stands on the post-training side: per-channel calibration recovers Sudoku with no retraining at all (§4.4). What survives compression and what repairs it together fix the configuration worth deploying.

4.7 Optimal Deployment Configuration

The deployment configurations are summarized in Table 7. We measure step latencies, puzzle latencies, and peak memory directly on physical hardware (Samsung Galaxy S24, Samsung Galaxy S22 5G, and Dragonwing RB3 Gen 2) via Qualcomm AI Hub (Table 6). No INT4 configuration was run on device: Table 6 reports FP32 and INT8 latency only, so the 3.26 MB INT4 configuration is a weights-only estimate throughout. INT8 with single-puzzle flash loading and reduced recursion (H=1, n_{sup}=8) occupies 6.51 MB of weights (analytical) and runs at ∼500 GFLOPs per ARC-AGI-1 puzzle. Weights alone are not the whole footprint: including activations and carry state, the ARC/Maze INT8 streaming peak is 9.51 MB against Sudoku's 3.06 MB (Table 5, Appendix A), so only Sudoku currently runs inside an 8 MB envelope, and the 4 MB target remains a weights-only projection.

Memory accounting. Weights alone, weights plus activations, the on-disk artifact and host-process RSS are different quantities. Table 4 gives the scope and provenance of each figure we report.

Maze-Hard is the cleanest deployable case: with no test-time augmentation and no embedding table, its per-pass latency is its per-puzzle latency, and naïve INT4 costs it 0.4 pp (Table 8).

Per-channel calibrated INT4 weights fit under the 4 MB MCU budget (3.26 MB, weights only) and calibration recovers near-FP32 accuracy with no retraining, demonstrated directly on Sudoku-Extreme and, on the official checkpoint, on ARC-AGI-1 (29.4% vs. 29.5% FP32, Table 3; §4.4). The practical compression floor is therefore calibrated INT4, not INT8; naïve INT4 remains unusable, and the fidelity diagnostic separates the two before deployment. The 6× FLOPs reduction also lowers modeled energy proportionally (∼3,000 to ∼500 mJ/puzzle, analytical; Appendix M), and there is no cheaper shortcut through the input: truncating the context destroys accuracy on every task—halving it collapses Maze and Sudoku to 0% puzzle exact (Appendix N).

5 Discussion

Why the architecture split exists. The two mixers spread their weights differently. Attention encodes sparse relational updates that tolerate coarse noise, whereas MLP-mixing encodes denser token-mixing whose wider per-channel ranges need per-channel scales to survive (Figure 4). Architecture choice in recursive models is therefore a deployment decision, best made at design time.

The microcontroller tier. We target a regime with kB-to-MB SRAM and no DRAM, where nothing reasoning-capable is deployed today. Mobile-NPU deployment of 1–7B-parameter LLMs is a complementary industrial problem with gigabytes of DRAM to work with, and a 3–7 MB recursive reasoner would be a first entry at our tier. This study establishes the compression recipe that gets a reasoner to that size with its reasoning intact (§3); the streaming peaks fit an 8 MB envelope only on Sudoku (Table 4).

Fidelity should generalize beyond TRM. Because carry-trajectory fidelity needs only a full-precision reference and the model's own hidden states (no labels, no task run) it should transfer to any iterative-latent system with accessible state:

Universal Transformers (Dehghani et al., 2019), deep equilibrium models, and state-space models.

Limitations

Our study centers on the TRM family across three tasks, with a second architecture (HRM) evaluated only on Sudoku-Extreme; broader generalization across architectures and tasks is untested. The calibrated-INT4 recovery is measured by full-system accuracy on Sudoku and, on the official checkpoint, on ARC (Table 3); the Sudoku TRM-MLP checkpoint is a community reproduction, so its absolute accuracies are not comparable to the original release (App. O). The fidelity diagnostic requires a full-precision reference, so it supports a pre-deployment decision rather than monitoring a live model. ARC accuracies are reported single-pass (no test-time augmentation); TRM's published protocol uses 1,000-augmentation voting, which would raise both accuracy and per-puzzle deployment cost proportionally. Maze and Sudoku need no augmentation and are the configurations whose latency is directly deployable. One boundary on the claim: every model here is recursive, so the results isolate what recursion does under compression but do not separate that from what being a few-million-parameter model does. A same-size autoregressive control would separate them, and no such checkpoint exists for these tasks.

References

Hamza A Abushahla, Dara Varam, Ariel Justine N Panopio, and Mohamed I AlHajri. 2025. Neural network quantization for microcontrollers: A comprehensive survey of methods, platforms, and applications. *arXiv preprint arXiv:2508.15008*.

MohammadHossein AskariHemmat, Reyhane Askari Hemmat, Alex Hoffman, Ivan Lazarevich, Ehsan Saboori, Olivier Mastropietro, Sudhakar Sah, Yvon Savaria, and Jean-Pierre David. 2022. QReg: On regularization effects of quantization. *arXiv preprint arXiv:2206.12372*.

MohammadHossein AskariHemmat, Ahmadreza Jeddi, Reyhane Askari Hemmat, Ivan Lazarevich, Alexander Hoffman, Sudhakar Sah, Ehsan Saboori, Yvon Savaria, and Jean-Pierre David. 2024. QGen: On the ability to generalize in quantization aware training. *arXiv preprint arXiv:2404.11769*.

Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. 2022. End-to-end algorithm synthesis with recurrent networks: Logical extrapolation without overthinking. In *Advances in Neural Information Processing Systems*, volume 35, pages 20232–20242.

Ondrej Bohdal, Konstantinos Theodosiadis, Asterios Mpatziakas, Dimitrios Filippidis, Iro Spyrou, Christos Zonios, Anastasios Drosou, Dimosthenis Ioannidis, Kyenghun Lee, Jijoong Moon, and 1 others. 2025. On-device system of compositional multi-tasking in large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 416–424.

François Chollet. 2019. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. 2019. Universal transformers. In *International Conference on Learning Representations*.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. LLM.int8(): 8-bit matrix multiplication for transformers at scale. *Advances in neural information processing systems*, 35:30318–30332.

Hanwen Du, Yuxin Dong, and Xia Ning. 2026. Latent thinking optimization: Your latent reasoning language model secretly encodes reward signals in its latent thoughts. In *International Conference on Learning Representations*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38:41340–41391.

Song Han, Huizi Mao, and William J Dally. 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations*.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. 2025. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*.

Saqib Javed, Hieu Le, and Mathieu Salzmann. 2025. QT-DoG: Quantization-aware training for domain generalization. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 26981–27004.

Jiachen Jiang, Jinxin Zhou, and Zhihui Zhu. 2025. Tracing representation progression: Analyzing and enhancing layer-wise similarity. In *International Conference on Learning Representations*, volume 2025, pages 1118–1143.

Alexia Jolicoeur-Martineau. 2025. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*.

Sehoon Kim, Amir Gholami, Zhewei Yao, Michael W Mahoney, and Kurt Keutzer. 2021. I-BERT: Integer-only BERT quantization. In *International conference on machine learning*, pages 5506–5518. PMLR.

Harsh Kohli, Srinivasan Parthasarathy, Huan Sun, and Yuekun Yao. 2026. Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers. *arXiv preprint arXiv:2604.07822*.

Zhen Li, Yupeng Su, Songmiao Wang, Runming Yang, Congkai Xie, Aofan Liu, Ming Li, Jiannong Cao, Yuan Xie, Ngai Wong, and 1 others. 2025. Quantization meets reasoning: Exploring and mitigating degradation of low-bit LLMs in mathematical reasoning. *arXiv preprint arXiv:2505.11574*.

Ji Lin, Wei-Ming Chen, Yujun Lin, Chuang Gan, Song Han, and 1 others. 2020. MCUNet: Tiny deep learning on IoT devices. *Advances in neural information processing systems*, 33:11711–11722.

Peiyu Liu, Zikang Liu, Ze-Feng Gao, Dawei Gao, Wayne Xin Zhao, Yaliang Li, Bolin Ding, and Ji-Rong Wen. 2024. Do emergent abilities exist in quantized large language models: An empirical study. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 5174–5190.

Ruikang Liu, Yuxuan Sun, Manyi Zhang, Haoli Bai, Xianzhi Yu, Tiezheng Yu, Chun Yuan, and Lu Hou. 2025. Quantization hurts reasoning? an empirical study on quantized reasoning models. In *Proceedings of the Conference on Language Modeling*. ArXiv:2504.04823.

Wenquan Lu, Yuechuan Yang, Kyle Lee, Yanshu Li, and Enqi Liu. 2025. Latent chain-of-thought? decoding the depth-recurrent transformer. *arXiv preprint arXiv:2507.02199*. COLM 2025 Workshop on LLM Explainability for Reasoning and Planning.

Keyu Lv, Manyi Zhang, Xiaobo Xia, Jingchen Ni, Shannan Yan, Xianzhi Yu, Lu Hou, Chun Yuan, and Haoli Bai. 2026. What makes low-bit quantization-aware training work for reasoning llms? a systematic study. *arXiv preprint arXiv:2601.14888*.

Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart Van Baalen, and Tijmen Blankevoort. 2021. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*.

Gauthier Tallec, Edouard Yvinec, Arnaud Dapogny, and Kevin Bailly. 2023. Fighting over-fitting with quantization for learning deep neural networks on noisy labels. In *2023 IEEE International Conference on Image Processing (ICIP)*, pages 575–579. IEEE.

Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. 2025. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*.

Michael S Zhang, Rishi A Ruia, Arnav Kewalram, Saathvik Dharmapuram, Utkarsh Sharma, and Kevin Zhu. 2025. When less is more: 8-bit quantization improves continual learning in large language models. *arXiv preprint arXiv:2512.18934*.

Zihan Zhou, Simon Kurz, and Zhixue Zhao. 2025. Revisiting pruning vs quantization for small language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12055–12070.

A Peak SRAM Memory Footprints

Figure	What it measures	Measured?	Reported in
6.7 MB	INT8 backbone weights	analytical	Table 8
6.51 MB	INT8 + flash, weights only	analytical	Table 7
3.26 MB	INT4 cal. + flash, weights only	analytical	Table 7
6.91 MB	on-disk ONNX artifact	measured	Table 6
9.51 MB	streaming peak, incl. activations	analytical	Table 5
115–226 MB	Android host-process RSS	measured	Table 6

Table 4: Scope and provenance of every memory figure this paper reports. The weights-only rows exclude activations and carry state; the RSS row is a host-process figure, not an SRAM footprint. No INT4 configuration was run on device.

For on-device edge deployment, we evaluate the peak memory footprint under two execution scenarios: a *Full Load* scenario (where the entire backbone and intermediate activations/carries are stored in SRAM) and a *Streaming* scenario (where only the current recursive step is loaded, leveraging single-puzzle flash loading of embedding tables).

Table 5 compiles these peak SRAM requirements against a theoretical 8 MB SRAM hardware budget limit.

Model / Variant	**Streaming**	**Full Load**	**Fits 8 MB? (Streaming)**
ARC (FP32, no embed)	27.31 MB	40.31 MB	✗
ARC (INT8, no embed)	9.51 MB	12.76 MB	✗
Maze (FP32)	27.31 MB	40.31 MB	✗
Maze (INT8)	9.51 MB	12.76 MB	✗
Sudoku (FP32)	11.08 MB	20.65 MB	✗
Sudoku (INT8)	3.06 MB	5.45 MB	✓

Table 5: Peak memory footprint (SRAM) for deployable ONNX models, including intermediate activations and carry state. Embedding layers are excluded (no-embed) as they are streamed via flash loading. The 8 MB target assumes streaming mode. These activation-inclusive peaks complement the weights-only analytical figures of Table 7.

Under streaming execution, only the Sudoku (INT8) model fits within the 8 MB SRAM budget (3.06 MB streaming peak); the ARC and Maze INT8 models peak at 9.51 MB and so exceed it (§4.7).

B On-Device Hardware Benchmarks

We compile on-device step and puzzle latencies, peak memory usage, and file sizes directly on three physical edge devices:

- **Samsung Galaxy S24**: Featuring the flagship Qualcomm Snapdragon 8 Gen 3 chipset (SM8650, Hexagon v75 NPU).
- **Samsung Galaxy S22 5G**: Featuring the Qualcomm Snapdragon 8 Gen 1 chipset (SM8450, Hexagon v70 NPU).
- **Dragonwing RB3 Gen 2**: Featuring the Qualcomm QCS6490 chipset (Hexagon NPU).

Evaluations are compiled and run via Qualcomm AI Hub targeting native hardware execution. Table 6 reports the full set of consolidated benchmark results.

Static INT8 quantization delivers substantial latency gains for compute-bound tasks with long sequences (ARC-AGI-1 and Maze-Hard) but incurs a step-latency penalty on the computationally light Sudoku-Extreme task, which is dominated instead by NPU driver and dispatch overhead and format-conversion casting; §B.1 works through why.

The controlled quantity is the per-*step* latency, which is depth-independent by construction: on ARC the FP32→INT8 per-step ratio is $40.20 \rightarrow 25.69$ ms ($1.56\times$). The FP32 and INT8 rows for ARC and Maze

Model / Variant	Device	H	n_{sup}	Step Lat. (ms)	Puzzle Lat. (ms)	Peak Mem. (MB)	Size (MB)	PExact
ARC (FP32‡)	Galaxy S24	3	16	40.20	643.18	185.24	27.56	0.355
ARC (FP32‡)	Galaxy S22	3	16	85.32	1365.18	180.73	27.56	0.355
ARC (FP32‡)	Dragonwing	3	16	1737.80	27804.82	127.11	27.56	0.355
ARC (INT8)	Galaxy S24	1	8	25.69	205.53	173.96	6.91	0.260
ARC (INT8)	Galaxy S22	1	8	55.59	444.69	226.49	6.91	0.260
ARC (INT8)	Dragonwing	1	8	672.89	5383.10	115.48	6.91	0.260
Maze (FP32‡)	Galaxy S24	3	16	119.83	1917.25	229.97	28.41	0.870
Maze (FP32‡)	Galaxy S22	3	16	224.45	3591.18	189.69	28.41	0.870
Maze (FP32‡)	Dragonwing	3	16	5239.83	83837.22	132.36	28.41	0.870
Maze (INT8)	Galaxy S24	1	8	25.71	205.72	170.73	6.90	0.850
Maze (INT8)	Galaxy S22	1	8	55.25	441.98	222.51	6.90	0.850
Maze (INT8)	Dragonwing	1	8	669.16	5353.26	112.21	6.90	0.850
Sudoku (FP32‡)	Galaxy S24	3	16	11.38	182.05	154.41	20.47	0.717
Sudoku (FP32‡)	Galaxy S22	3	16	30.17	482.75	179.78	20.47	0.717
Sudoku (FP32‡)	Dragonwing	3	16	606.02	9696.40	58.64	20.47	0.717
Sudoku (INT8)	Galaxy S24	3	16	27.23	435.60	160.41	5.10	0.699
Sudoku (INT8)	Galaxy S22	3	16	79.68	1274.86	210.44	5.10	0.699
Sudoku (INT8)	Dragonwing	3	16	334.68	5354.94	57.25	5.10	0.699

Table 6: On-device hardware benchmark results compiled via Qualcomm AI Hub. All evaluations are run on physical devices: Galaxy S24 (Snapdragon 8 Gen 3), Galaxy S22 (Snapdragon 8 Gen 1), and Dragonwing (Qualcomm QCS6490). ‡FP32 rows were compiled with `enable_htp_fp16_precision=1` and execute as FP16 on the Hexagon vector engine (HVX); see §B.1. Peak Mem. is the Android process RSS reported by AI Hub (application heap + runtime + RPC buffers); it is a host-process figure on these DRAM-backed devices and is not an SRAM footprint (see the memory-accounting taxonomy in §4.7).

run at different recursion depths (H=3, n_{sup}=16 against H=1, n_{sup}=8; the depth columns make this explicit), so their per-*puzzle* ratio combines precision with depth. The Sudoku rows are depth-matched end to end (both H=3, n_{sup}=16), and there INT8 is slower; §B.1 explains why.

Notebook and on-device harnesses. The on-device INT8 run scores 26.0% for the H=1, n_{sup}=8 ARC configuration against 35.25% in the notebook harness, both single-pass. The quantizer accounts for the gap. At FP32 the two harnesses agree to 0.5 pp (35.5% on device, 36.0% in harness); they diverge by 9.3 pp only once INT8 is applied, where `bitsandbytes` and static QDQ differ. Attributing the residual gap to a specific implementation detail of one quantizer is left to future work.

B.1 Sequence-Length Dependent Quantization Penalty

All rows in Table 6 were compiled through the Qualcomm AI Hub QNN execution provider (`libQnnHtp.so`) targeting the Hexagon processor; no row falls back to CPU execution. Compilation used `enable_htp_fp16_precision=1`, so graphs labeled "FP32" execute in FP16 on the Hexagon vector engine (HVX), while INT8 graphs execute on the HTP tensor units with quantize/dequantize (QDQ) boundaries at node interfaces.

The benchmark reveals a counter-intuitive result: while static INT8 quantization accelerates the ARC-AGI-1 and Maze-Hard models, it actually leads to a latency slowdown for the Sudoku-Extreme model per step (from 11.38 ms to 27.23 ms on the S24). This behaviour is a classic *small-matrix INT8 penalty* governed by the sequence length (T):

- **Compute-Bound Regime (ARC & Maze):** ARC and Maze operate on a sequence length $T = 900$. At this scale, the weight projection matrices are large (900×512 by 512×2048), which translates to heavy arithmetic compute ($\sim$187 GFLOPs/step). Moving this workload from the HVX FP16 pipeline to the HTP's INT8 tensor units reduces execution time significantly, easily overriding the fixed dispatch and QDQ overhead.

- **Overhead-Bound Regime (Sudoku):** Sudoku has a sequence length of only $T = 81$ (97 with padding and embeddings). The matrix multiplications are tiny (e.g., 97×512 by 512×2048), requiring only

$\sim$25 MFLOPs/step (7,500$\times$ less compute than Maze). A 97×512 matrix multiplication requires only $\sim$100K multiply-accumulates, which is too small to saturate the INT8 parallel execution blocks. Consequently, the fixed overhead of the Quantize/Dequantize (QDQ) operators—which convert tensors back and forth at every node boundary—dominates the execution time. While the FP16-on-HVX model runs in a single, unbroken compiler-fused pipeline in 11.38 ms, the INT8 model is penalized by a constant $\sim$25 ms invocation and marshalling overhead.

Sudoku also relies on token-mixing MLP layers (`mlp_t` layers of 388 KB/194 KB). These represent extremely small matrices. The Qualcomm Hexagon tensor processor has a minimum efficient tile size threshold; matrix dimensions below this limit fail to trigger accelerated execution paths and fall back to slower scalar execution, further exacerbating the INT8 penalty.

The crossover point where static INT8 quantization becomes beneficial for this recursive transformer architecture on mobile chipsets is approximately $T \approx 200$–300. Below this threshold, FP16-on-HVX execution remains faster.

B.2 Static vs. Dynamic Quantization

Dynamic quantization preserves full-precision accuracy on every task. Static quantization does not, and Sudoku-Extreme recovers only under selective operator exclusion: leaving activation and carry-state updates in FP32 and quantizing only the `MatMul`/`Gemm` weights. The trade-off matters for deployment: dynamic quantization is the robust out-of-the-box path for recursive models, but NPU accelerators require static, pre-allocated tensor boundaries, so on-device execution inherits the static path and its operator exclusions (§B.1). We report this trade-off qualitatively; per-task accuracy under the static path was not separately measured.

C Full Precision × Task Results

Configuration	Weights	GFLOPs/puzzle	Fits 8 MB (weights)?
FP32, H=3, n=16, full emb.	$\sim$126 MB	3,000	✗
INT8, H=1, n=8, flash loading	**6.51 MB**	**500**	✓
INT4 calibrated, H=1, n=8, flash	**3.26 MB**	**500**	✓

Table 7: Deployment configurations per ARC-AGI-1 puzzle. Analytical, weights-only; excludes activations and carry state (see Table 5 for activation-inclusive streaming peaks). Bold marks viable targets. Figures show backbone-only weight footprint (ARC embedding handled via flash loading). On-device latencies are measured directly on physical hardware (Table 6). Per-channel calibrated INT4 recovers near-FP32 accuracy without retraining (§4.4), making the 3.26 MB configuration a candidate 4 MB MCU target.

Table 8 reports full-system accuracy across all precision levels and tasks at the evaluated recursive depth.

		ARC-AGI-1		Maze-Hard		Sudoku-Extreme	
Precision	**SRAM**	*PExact*	*CellAcc*	*PExact*	*CellAcc*	*PExact*	*CellAcc*
FP32†	26.7 MB	36.00%	88.22%	86.80%	99.52%	69.10%	87.47%
FP16	13.3 MB	36.00%	87.57%	87.00%	99.53%	68.50%	87.20%
INT8	6.7 MB	36.00%	87.62%	87.00%	99.53%	69.10%	87.51%
INT4 naïve	3.3 MB	25.50%	84.80%	86.40%	99.50%	5.30%	66.02%

Table 8: Full-system precision × task accuracy results at the evaluated recursive depth, *secondary protocol* (H=3, n_{sup}=10, bf16 forward, first-N subsets); the primary-protocol headline figures are in Table 1, and the official-checkpoint ARC figures in Table 3. ARC-AGI-1 public evaluation set (400 tasks), single-pass (no test-time augmentation). SRAM shows backbone-only footprint (ARC embedding handled via flash loading). †FP32 rows executed a bf16 forward pass; SRAM at FP32 precision.

D Detailed Recursive Sweep Grids

We report the complete multi-variant validation sweeps across combinations of recursive steps (n) and precision modes on GPU (CUDA) for all three architectures. Tables 9, 10, and 11 present these complete grids, including exact match accuracy and cell accuracy.

E Depth × Quantization Sweep (ARC-AGI-1)

Table 12 reports key ARC-AGI-1 depth configurations (full-system, single-pass): INT8 matches FP32 at every depth tested.

F Depth × Quantization Sweep (Maze-Hard)

Table 13 shows key configurations from the Maze-Hard depth sweep, confirming rapid saturation.

G Depth × Quantization Sweep (Sudoku-Extreme)

Table 14 shows key configurations from the Sudoku-Extreme depth sweep: accuracy continues to improve with depth within the swept range ($n_{\text{sup}} \leq 10$).

H Architecture Ablation: MLP-Mixing vs. Attention

To separate architecture from task precision we evaluate three recursive architectures on the *same* task (Sudoku-Extreme) under an identical harness (N=256, 16 ACT steps, no TTA; Table 15). TRM-MLP-Mixing and TRM-Attention are the same model family at the same full-precision accuracy (73.8 vs. 73.1% puzzle exact), differing only in the token mixer; HRM (Wang et al., 2025) is a different recursive architecture (separate high- and low-level modules, attention-based). Under naïve INT4, TRM-MLP-Mixing collapses to 10.2% while both attention architectures retain their accuracy. Since TRM-Attention matches TRM-MLP at full precision, this is not an accuracy-headroom effect—the INT4 fragility is specific to MLP-mixing, not the task. The carry-trajectory fidelity diagnostic grades all three correctly (0.35 / 0.87 / 0.98), confirming it generalizes across recursive architectures.

I Carry-State Diagnostic: Full Data

We measure two carry-state quantities, computed by hooking the shared reasoning module across all recursive calls within one inference pass. Both are computed on real test puzzles (Maze N=64, ARC N=32, Sudoku N=256); the ARC and Maze checkpoints are the public TRM releases, the Sudoku TRM-MLP checkpoint is a community reproduction (see checkpoint provenance, §O), and all numbers below are reproduced from those checkpoints.

Consecutive carry similarity (single-model). The mean cosine similarity between consecutive carry states (Table 16) is task-specific but *insensitive* to compression: it is essentially identical under FP32, INT8, and INT4 on every task (changes ≤ 0.064), and is therefore a weak failure signal. We report it for completeness; the diagnostic below is the one we recommend.

Carry-trajectory fidelity (vs. full-precision reference). The recommended diagnostic is the cosine similarity between the quantized model's final carry state and the FP32 model's final carry state on the same input—how far quantization pushes the reasoning trajectory off the full-precision path (Table 17). It requires only the FP32 reference (available before deployment) and no task labels. Unlike the consecutive signal, fidelity is *graded and monotonic with severity*: it stays ≈1.0 wherever INT4 is harmless (Maze, all INT8) and falls in proportion to the accuracy damage—moderately on ARC (0.69, −10 pp) and catastrophically on Sudoku (0.35, −64 pp). Per-channel calibrated INT4 *recovers* fidelity in lockstep with accuracy, so the diagnostic predicts both failure and recovery.

Calibrated INT4 recovers Sudoku. Measured under an identical evaluation harness (N=256, 16 ACT steps, no TTA), naïve per-tensor INT4 collapses Sudoku to 10.2% puzzle-exact while per-channel calibrated INT4 recovers to 71.9% (vs. 73.8% FP32 under the same harness)—i.e., the catastrophic INT4 result is specific to naïve quantization, and the 4 MB MCU target is reachable with calibration (Table 18).

Precision	H	**Metric**	$n=1$	$n=2$	$n=4$	$n=8$	$n=16$
FP32 (bf16)	3	Exact	0.2850	0.3600	0.3575	0.3575	0.3600
		Cell	0.8762	0.8839	0.8764	0.8760	0.8757
	1	Exact	0.0025	0.0750	0.3425	0.3575	0.3550
		Cell	0.5410	0.7954	0.8825	0.8832	0.8762
FP16	3	Exact	0.2850	0.3600	0.3575	0.3600	0.3625
		Cell	0.8762	0.8838	0.8765	0.8763	0.8755
	1	Exact	0.0025	0.0750	0.3450	0.3550	0.3575
		Cell	0.5410	0.7953	0.8825	0.8831	0.8773
INT8 (bnb)	3	Exact	0.2850	0.3575	0.3575	0.3600	0.3600
		Cell	0.8760	0.8837	0.8827	0.8842	0.8670
	1	Exact	0.0025	0.0750	0.3425	0.3525	0.3625
		Cell	0.5415	0.7953	0.8827	0.8835	0.8767
INT4 (naive)	3	Exact	0.1575	0.2700	0.2675	0.2625	0.2575
		Cell	0.8372	0.8626	0.8473	0.8430	0.8322
	1	Exact	0.0000	0.0225	0.2525	0.2650	0.2600
		Cell	0.3954	0.7148	0.8582	0.8494	0.8415

Table 9: Detailed recursive sweep grid for ARC-AGI-1 (seq_len = 900) across H (outer cycles), n (supervision steps), and precision variants.

Precision	H	**Metric**	$n=1$	$n=2$	$n=4$	$n=8$	$n=16$
FP32 (bf16)	3	Exact	0.7850	0.8670	0.8700	0.8700	0.8700
		Cell	0.9940	0.9953	0.9953	0.9953	0.9953
	1	Exact	0.0000	0.1390	0.8440	0.8690	0.8700
		Cell	0.4363	0.9808	0.9949	0.9953	0.9953
FP16	3	Exact	0.7850	0.8670	0.8700	0.8700	0.8700
		Cell	0.9940	0.9953	0.9953	0.9953	0.9953
	1	Exact	0.0000	0.1380	0.8440	0.8700	0.8700
		Cell	0.4362	0.9808	0.9949	0.9953	0.9953
INT8 (bnb)	3	Exact	0.7830	0.8680	0.8710	0.8700	0.8700
		Cell	0.9940	0.9953	0.9953	0.9953	0.9953
	1	Exact	0.0000	0.1380	0.8450	0.8690	0.8700
		Cell	0.4333	0.9808	0.9949	0.9953	0.9953
INT4 (naive)	3	Exact	0.7660	0.8610	0.8640	0.8640	0.8640
		Cell	0.9934	0.9949	0.9950	0.9950	0.9950
	1	Exact	0.0000	0.1580	0.8360	0.8630	0.8630
		Cell	0.4734	0.9805	0.9945	0.9950	0.9950

Table 10: Detailed recursive sweep grid for Maze-Hard (seq_len = 900) across H (outer cycles), n (supervision steps), and precision variants.

J Structured Pruning and Knowledge Distillation

Structured pruning. Magnitude-based structured pruning was evaluated at 25% and 50% sparsity targets across all three tasks (Table 19).

Knowledge distillation. Student models (hidden=256, 1 layer, 1-cycle) were trained via KL-divergence distillation from the full teacher (Table 20).

K Linear-Attention Approximation (Maze)

Replacing softmax attention with ELU feature-map linear attention ($\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$) on Maze illustrates the same cell-vs-puzzle split as quantization: the swap preserves token-level accuracy but destroys puzzle-level reasoning. Evaluated at H=3, n_{sup}=1 (hence the 78.50% baseline rather than the full-depth 86.80%): zero-shot, cell accuracy holds at 87.3% while puzzle-exact falls to 0.00% (from 78.50% / 99.40%); brief retraining (200 steps) recovers cell accuracy (87.5%) but not puzzle-exact (0.00%). This is a third intervention—alongside pruning and distillation—under which local prediction survives while global constraint satisfaction collapses. Sudoku-Extreme (MLP-mixing) has no attention layers and is not applicable.

L QAT Configurations

Table 21 reports the six quantization-aware-training configurations of §4.6: two objectives at three learning rates each, 1,000 steps at batch 32 from the calibrated-INT4 initialization (72.3% at step 0).

Final pass@1 (1,000 steps)	lr 10^{-5}	lr 10^{-4}	lr 10^{-3}
Cross-entropy	53.9%	32.4%	2.3%
Distillation (KD)	53.9%	37.1%	7.8%

Table 21: QAT from the calibrated-INT4 initialization (72.3% at step 0). All six configurations degrade the model they start from.

M Energy per Inference

Estimated energy follows the FLOPs counts under a Cortex-M55 power model (∼1 mJ/GFLOP): ARC-AGI-1 at full depth (∼3,000 GFLOPs/puzzle) draws ∼3,000 mJ/puzzle, versus ∼500 mJ at the reduced configuration (H=1, n_{sup}=8). Sudoku-Extreme (∼256 GFLOPs at default depth) draws ∼256 mJ/puzzle. These are analytical estimates under the same FLOPs-based model. The model is precision-blind—it assigns no separate energy-per-operation term to FP32 vs. INT8—so the 6× energy saving follows entirely from the 6× FLOPs (cycle) reduction, not from quantization.

N Context-Length Truncation

Because attention cost scales as $\mathcal{O}(T^2)$, we tested zero-padding the input beyond a truncated context. Truncation is destructive on every task (Table 22): halving the sequence length takes Maze and Sudoku to zero and costs ARC nearly ten points. There is no cheap context shortcut—the full window is load-bearing.

Context	**ARC**	**Maze**	**Sudoku**
full (900 / 81)	36.00	78.50	28.40
∼50%	26.50	0.00	0.00
∼25%	10.75	—	—

Table 22: Puzzle-exact accuracy under input context truncation, evaluated at H=3, n_{sup}=1 (the full-context rows match the n=1 column of Tables 10–11, which is why they sit below the full-depth baselines quoted elsewhere). Maze and Sudoku collapse to 0 at 50%; ARC degrades severely.

O Implementation Details

Checkpoint provenance. The Sudoku TRM-MLP checkpoint is the alphaXiv community reproduction of TRM, whose model card reports 79.37% puzzle exact vs. 87.4% in the original paper. Our Sudoku claims are therefore relative comparisons between compression variants evaluated at matched checkpoints, not absolute-accuracy claims against the original release.

Quantization. INT8 and INT4 PTQ were applied using `bitsandbytes` v0.41 (Dettmers et al., 2022) with `load_in_8bit=True` and `load_in_4bit=True` respectively. Per-channel calibration for INT4 calibrated used 128 calibration puzzles drawn from the training set.

Carry-state extraction. s_h was computed by registering a forward hook on the carry-state update function g_ϕ to capture Z_h at each outer cycle. Cosine similarity was computed token-wise and averaged across the sequence dimension and across all puzzles in the evaluation set.

QAT. The headline QAT runs (§4.6) initialize from the calibrated-INT4 checkpoint and fine-tune for 1,000 steps at batch 32, under cross-entropy and token-distillation objectives at learning rates 10^{-5}, 10^{-4}, and 10^{-3}, with the deep-supervision ACT loop. A shorter probe (100 steps, lr=10^{-5}, cross-entropy, batch 4, from the naïve INT4 checkpoint) is the held-out run cited in §4.6. Training and evaluation used the Modal cloud platform.

ONNX export and edge deployment. Models were exported to ONNX format using `torch.onnx.export` for deployment via ONNX Runtime and Qualcomm AI Hub. The ONNX export produces a single-step model (`TRMBackboneStep`) that executes one H-cycle step, with the outer recursion loop managed by the inference harness.

Precision	H	Metric	$n=1$	$n=2$	$n=4$	$n=8$	$n=16$
FP32 (bf16)	3	Exact	0.2840	0.4960	0.6040	0.6690	0.7090
		Cell	0.7640	0.8147	0.8474	0.8680	0.8810
	1	Exact	0.0060	0.0210	0.3830	0.5470	0.6370
		Cell	0.5527	0.6943	0.7856	0.8293	0.8563
FP16	3	Exact	0.2830	0.4960	0.6100	0.6700	0.7160
		Cell	0.7639	0.8151	0.8488	0.8677	0.8851
	1	Exact	0.0060	0.0210	0.3830	0.5480	0.6390
		Cell	0.5526	0.6941	0.7856	0.8303	0.8574
INT8 (bnb)	3	Exact	0.2800	0.5010	0.6210	0.6790	0.7120
		Cell	0.7643	0.8183	0.8524	0.8721	0.8816
	1	Exact	0.0060	0.0210	0.3820	0.5540	0.6560
		Cell	0.5528	0.6942	0.7852	0.8306	0.8638
INT4 (naive)	3	Exact	0.0210	0.0290	0.0510	0.0550	0.0610
		Cell	0.6277	0.6490	0.6524	0.6573	0.6597
	1	Exact	0.0000	0.0120	0.0210	0.0370	0.0510
		Cell	0.5612	0.6097	0.6375	0.6536	0.6555

Table 11: Detailed recursive sweep grid for Sudoku-Extreme (seq_len = 81) across H (outer cycles), n (supervision steps), and precision variants.

Config	Precision	PExact	GFLOPs
H=3, n=2 (peak)	FP32	36.00%	375
H=1, n=4	INT8	34.25%	250
H=1, n=8	INT8	35.25%	500
H=1, n=16	INT8	36.25%	1,000
H=1, n=4	INT4	25.25%	250

Table 12: ARC-AGI-1 depth–precision sweep (full-system, single-pass, no TTA). INT8 at H=1, n_{sup}=8 reaches 35.25% at 500 GFLOPs—within 0.75 pp of the 36.00% FP32 peak at 6× fewer FLOPs.

Config	Prec.	PExact	GFLOPs
H=3, n=10†	FP32	86.80%	1,875
H=1, n=8	INT8	86.90%	500
H=1, n=4	INT8	84.50%	250
H=1, n=8	INT4	86.30%	500

Table 13: Maze-Hard depth–precision sweep. †Evaluation configuration (n_{sup}=10); TRM's default test-time setting is n_{sup}=16. INT8 at H=1, n=8 matches the evaluated FP32 configuration at 3.75× fewer FLOPs.

Config	Prec.	PExact	GFLOPs
H=3, n=10†	FP32	69.10%	256
H=4, n=10	FP32	70.30%	342
H=4, n=10	INT8	70.20%	342
H=1, n=8	INT8	55.40%	68
H=4, n=10	INT4	6.00%	342

Table 14: Sudoku-Extreme depth–precision sweep. †Evaluation configuration (n_{sup}=10); TRM's default test-time setting is n_{sup}=16. INT8 at H=4, n=10 matches FP32 (70.20% vs 70.30%); reducing depth to H=1 causes major accuracy loss. Full depth is genuinely needed.

Architecture (Sudoku)	FP32	INT4	Fidelity
TRM-MLP-Mixing	73.8	10.2	0.35
TRM-Attention	73.1	69.5	0.87
HRM (diff. family)	47.7	48.4	0.98

Table 15: Same task (Sudoku-Extreme), three recursive architectures (secondary protocol; primary-protocol values for the two TRM variants are in Table 1): naïve INT4 puzzle-exact (%) and final-state carry fidelity. Only MLP-mixing collapses; both attention architectures—including a different model family (HRM)—survive. TRM-Attention matches TRM-MLP at FP32, ruling out an accuracy-headroom confound. HRM is evaluated on Sudoku only and at a lower FP32 baseline, so it corroborates the pattern rather than serving as a controlled comparison.

Precision	ARC	Maze	Sudoku
FP32	0.415	0.697	0.806
INT8	0.412	0.697	0.806
INT4	0.479	0.699	0.841

Table 16: Mean consecutive carry-state cosine similarity. The change under INT4 is small on every task (≤ 0.064, and ≤ 0.035 on the two tasks where INT4 is catastrophic); the signal does not reliably separate compression that preserves reasoning from compression that destroys it. (Maze/Sudoku reproduced; ARC from the source run.)

Precision	Maze	ARC	Sudoku
FP32	1.000	1.000	1.000
INT8	1.000	0.999	0.998
INT4 naïve	0.989	0.688	0.353
INT4 calibrated	—	0.944	0.894
INT4 accuracy Δ	-0.4*pp*	-10.5*pp*	-63.8*pp*

Table 17: Carry-trajectory fidelity (final-state cosine vs. FP32), secondary protocol (the accuracy deltas match Table 8). Monotonic with INT4 severity (Maze $\rightarrow$ ARC $\rightarrow$ Sudoku); INT8 is never flagged; calibrated INT4 recovers both fidelity and accuracy. Last row: INT4-naïve puzzle-exact change vs. FP32.

Variant	PExact	Cell	Fidelity
FP32	73.8	88.9	1.000
INT8	73.1	88.7	0.998
INT4 naïve	10.2	67.8	0.353
INT4 calibrated	71.9	88.2	0.894

Table 18: Sudoku-Extreme under an identical harness (secondary protocol, N=256, no TTA; FP32 reads 73.8 here vs. 69.1 on the full test set elsewhere, and 74.6 under the primary protocol of Table 1). Fidelity tracks accuracy across all four variants.

	Dense (0%)		25% sparse		50% sparse	
Task	*PExact*	*CellAcc*	*PExact*	*CellAcc*	*PExact*	*CellAcc*
ARC	36.00%	87.55%	0.00%	0.27%	0.00%	0.58%
Maze	86.80%	99.52%	0.00%	86.40%	0.00%	0.00%
Sudoku	69.10%	87.47%	0.00%	50.01%	0.00%	37.81%

Table 19: Structured pruning results. Even 25% sparsity destroys puzzle-level reasoning across all tasks. Cell accuracy partially survives on Maze (86.40%) and Sudoku (50.01%), indicating that token prediction survives while global constraint satisfaction does not.

Task	Teacher	Student	PExact	CellAcc
Maze	6.82M	855K	0.00%	87.25%
Sudoku	5.03M	745K	0.00%	54.77%

Table 20: Knowledge distillation results. Student models (12–15% of teacher params) learn token-level patterns but fail entirely at puzzle-level reasoning; the recursive structure is functionally necessary.